

Observations from Observing zk-Proving *Proving Ethereum in real time*

What I built

Proofessor

Follows a standard Beacon API and requests proofs via zkboost

Persists per-proof history and reconciles pending work

Shows witness, queue and proving time

Proving stack improvements

Fail-fast configuration and proof tests

Stage timings and trace propagation in zkboost

Mainnet deployments and failure analysis

Making the fleet recover

21.5 hours stuck

A cancellation race wedged the zisk workers

Services stayed active while usable capacity disappeared

Queue wait exposed the lost capacity

72 seconds to recover

Added recovery timeout and restart handling

Enforced a minimum worker capacity

Validated recovery in a later mainnet run

27 hours of live mainnet proving

OpenVM 2.0.0 via axiom-edge

16 RTX 5090 GPUs across four hosts. July 28–29 run.

99.4%

of real blocks proven
within the 12s budget

6.26s

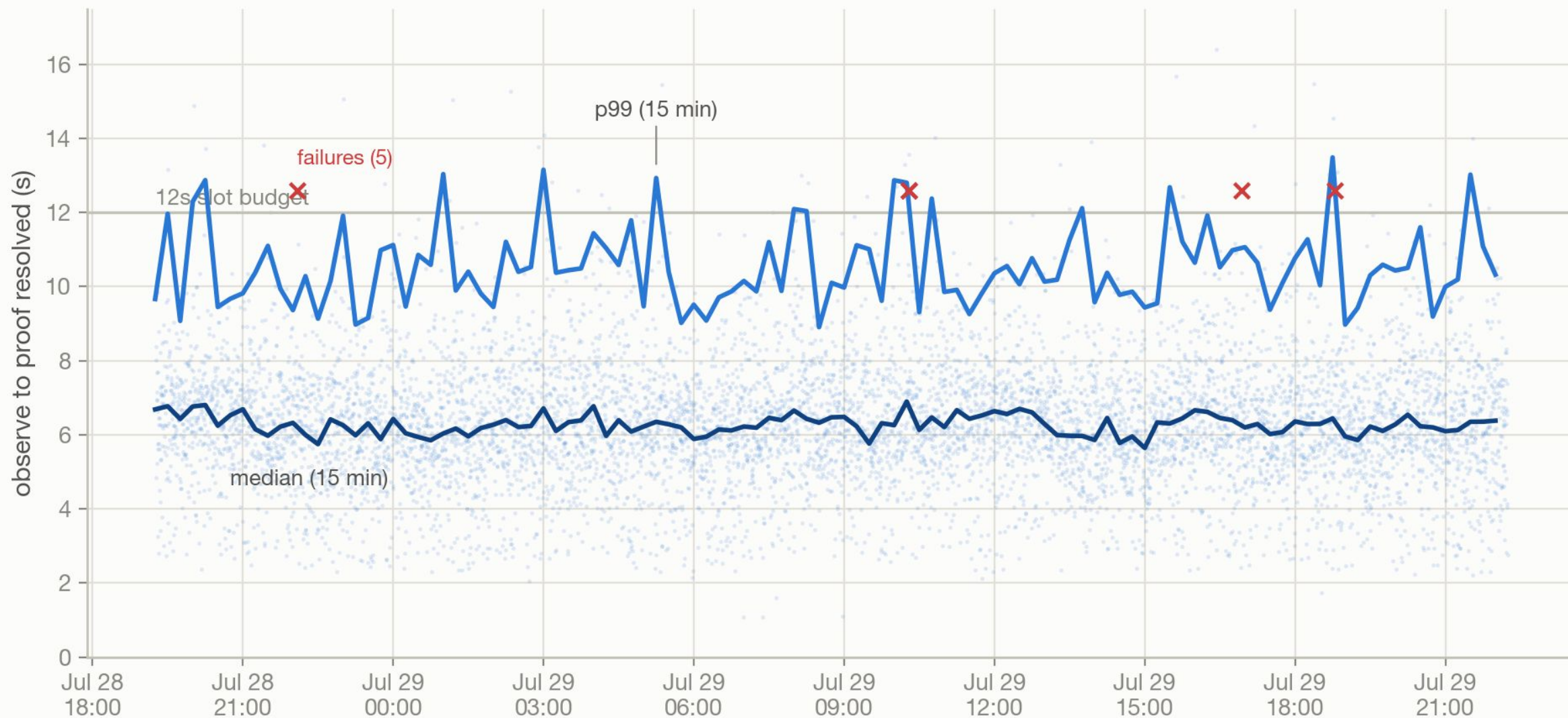
median end-to-end
latency

11.15s

p99 end-to-end
latency

openvm 2.0.0 streaming mainnet for 27 hours

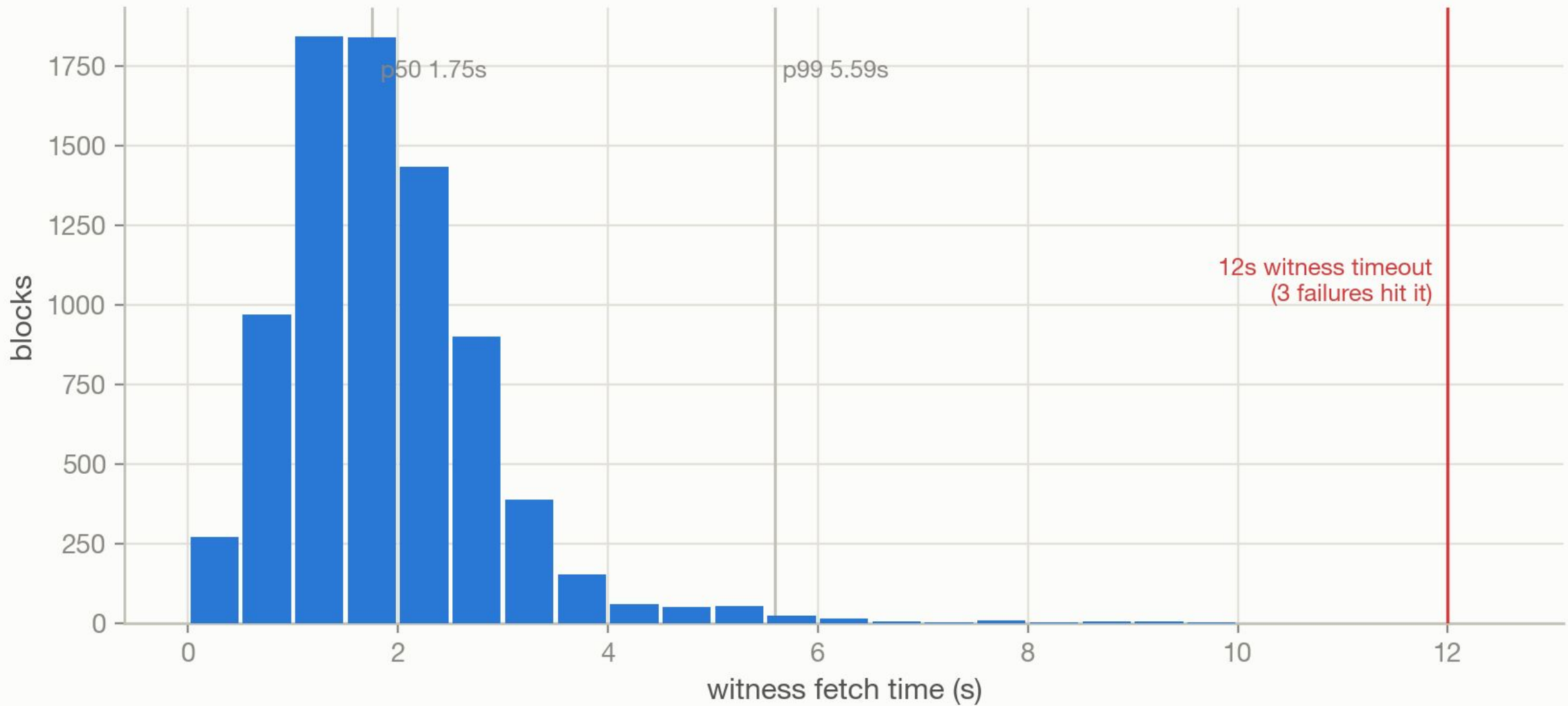
8,067 proofs over slots 14,870,178-14,878,270 (Jul 28 19:16 - Jul 29 22:14 UTC). One dot per block, end to end.



Timing starts at block observation. Five recorded failures.

Witness fetch is the tail: distribution over the 27h openvm run

8,067 proven blocks. Spikes above 6s are near-Poisson (2.3/hour) and cause the only real-time misses.



Among completed proofs, slow witness fetches caused the latency overruns.

What I would work on next

Reliable coverage

Account for every block

Persist verification
outcomes

Witness delivery

Reduce slow fetches

Add endpoint failover

Shared observability

Link blocks to prover
phases and workers

Compare zkVMs on the full
lifecycle

Questions?

Thank you

Parithosh, Stefan, Barnabas and the ethPandaOps team

Han, Ignacio and Kev

ethpandaops.io/docs/tooling/professoor